MOV 寄存器相关代码题专项复习

面向微机原理 / 8051 / Cortex-M3-M4：以代码判读、寄存器跟踪、寻址方式辨析为主

使用建议	先独立完成题目部分，再翻到后面的详解；遇到 MOVX/MOVC/MOVW/MOVT 时务必写出“数据从哪里来、到哪里去”。
重点比例	8051 数据传送与寄存器银行、Cortex-M 常数装载与寄存器/内存区别、接口类初始化代码。
常考陷阱	MOV 不等于访存万能指令；立即数范围、直接/间接寻址、SFR 与内部 RAM、标志位是否改变，经常混在代码跟踪题里。
资料依据	Keil/Arm 8051 Instruction Set Manual；Arm Cortex-M3/M4 instruction summary；Arm 官方 constant loading 说明。

一、MOV 家族考点速查

本页不是背诵表，而是做代码跟踪题时的“判题规则”。看见 MOV，先判断地址空间：寄存器、内部 RAM、SFR、代码存储器、外部数据存储器，不能只看助记符。

写法	数据流	考试提醒
8051: MOV A,#data	立即数 -> A	不改普通标志位；A 是大多数运算入口。
8051: MOV direct,#data	立即数 -> 内部 RAM/SFR 直接地址	direct >= 80h 时通常是 SFR，和 @Ri 间接寻址不是同一空间。
8051: MOV A,@R0 / @R1	内部 RAM 间接地址 -> A	R0/R1 内容是地址；@R2 不合法。
8051: MOV direct,direct	源 direct -> 目的 direct	汇编写法目的在前；机器码里 direct 字节顺序常让人误判。
8051: MOV DPTR,#data16	16 位立即数 -> DPTR	DPTR = DPH:DPL，常配合 MOVC/MOVX。
8051: MOV C,bit / MOV bit,C	位地址 <-> 进位位 C	这是位传送，C 会被目标 bit 覆盖或写到目标 bit。
8051: MOVC A,@A+DPTR	代码存储器查表 -> A	A 先参与形成偏移，执行后被表项覆盖。
8051: MOVX A,@DPTR	外部数据存储器 -> A	常与并行扩展、外设端口映射放在综合题。
Cortex-M: MOV Rd,Rm	寄存器 -> 寄存器	不是内存访问；[Rn] 才表示访问内存。
Cortex-M: MOVS Rd,#imm	小立即数 -> 低寄存器	常更新 N/Z；普通 MOV 通常不用于考标志更新。
Cortex-M: MOVW/MOVT	装入低/高 16 位	MOVW 会清高 16 位，MOVT 保留低 16 位。顺序很关键。

Cortex-M: LDR Rd,=const	伪指令	可能生成 MOV/MOVW/MOVT，也可能从 literal pool 读常数。

二、试卷里常用的深挖角度

- 合法性判断：如 MOV R2,R3、MOV @R0,@R1、MOV DPTR,A、MOV A,@R2 等，要先判断指令集是否提供这种寻址组合。
- 寄存器银行：R0-R7 是名字，不是固定物理地址；PSW.RS1:RS0 选择当前银行。直接地址 00h-1Fh 则是物理内部 RAM。
- SFR 与内部 RAM：direct 90h 常表示 P1 等 SFR；@R0 取 90h 时不是同一个“P1 直接地址”概念。
- 源目的顺序：8051 和 ARM 汇编通常都是目的在前，但 8051 某些机器码编码顺序容易造成“手译机器码”题误判。
- 常数装载：Cortex-M 的 32 位寄存器不代表 MOV 能塞任意 32 位立即数；大常数常用 MOVW/MOVT 或 LDR =const。
- 标志位：8051 的 MOV 基本不改算术标志；Cortex-M 的 MOVS 关注 N/Z。题干若问条件跳转结果，先检查有没有 S 后缀。
- 接口初始化：端口/控制寄存器通常是“地址装入寄存器 -> 常数装入寄存器 -> STR/MOVX 写出”的模板，MOV 本身多半只负责准备值。

三、题目部分

> 建议做题时每题都画一个极简跟踪表：寄存器列、直接地址列、间接地址列、标志位列。只在脑内硬算，很容易把 direct 和 @Ri 混掉。

A 组：8051 MOV / MOVC / MOVX 代码跟踪

A1. 8051 基础跟踪：立即数、直接地址与累加器

```
; 初始：30H=12H, 31H=34H, A=00H
MOV A,#56H
MOV 30H,A
MOV A,31H
MOV 31H,#78H
MOV 32H,A
```

要求：写出执行后 A、30H、31H、32H 的值。

A2. 8051 寄存器银行：R0 名字背后的物理地址

```
; 初始：PSW=08H，08H=11H，09H=22H，00H=AAH，01H=BBH
MOV R0,#30H
MOV R1,#31H
MOV 00H,#55H
MOV 08H,#66H
```

要求：指出当前 R0、R1 的值分别是多少；同时写出物理地址 00H、08H 的值。

A3. 8051 间接寻址：@R0 与 direct 的区别

```
; 初始：R0=40H，40H=12H，41H=34H，A=00H
MOV A,@R0
MOV 41H,A
MOV @R0,#56H
MOV A,40H
```

要求：写出 A、40H、41H 的最终值，并说明 @R0 的含义。

A4. 8051 指令合法性判断

```
① MOV R2,R3
② MOV A,R3
③ MOV @R0,@R1
④ MOV 35H,@R1
⑤ MOV DPTR,#1234H
⑥ MOV DPTR,A
⑦ MOV C,20H
⑧ MOV 20H,C
```

要求：判断每条是否合法；非法的给出最小改写思路。

A5. 8051 direct,direct 源目的顺序

```
; 初始：20H=11H，21H=22H
MOV 20H,21H
MOV 22H,20H
```

要求：执行后 20H、21H、22H 分别是多少？补一句为什么这题容易错。

A6. 8051 位传送与 C 标志

```
; 初始：C=0，位地址 20H=1，位地址 21H=0
MOV C,20H
MOV 21H,C
MOV C,21H
MOV 20H,C
```

要求：写出最终 C、位 20H、位 21H。

A7. 8051 DPTR 与查表 MOVC

```
; 假设代码存储器中：
; 2000H：3AH，2001H：7CH，2002H：55H，2003H：A6H
MOV DPTR,#2000H
MOV A,#02H
MOVC A,@A+DPTR
```

要求：写出最终 DPTR 和 A。说明 MOVC 使用的是哪个地址空间。

A8. 8051 MOVX 与外部数据空间

```
; 外部数据存储器：XDATA[8000H]=5AH
; 初始：A=00H，DPTR=0000H
MOV DPTR,#8000H
MOVX A,@DPTR
MOV 30H,A
```

要求：写出 A、DPTR、内部 RAM 30H。说明 MOVX 与 MOV 的空间差异。

A9. 8051 SFR 直接地址与端口写入

```
; 假设 90H 为 P1 端口 SFR，初始 P1=FFH
MOV 90H,#0F0H
MOV A,90H
MOV 30H,A
```

要求：按直接地址写法分析：P1、A、30H 最终是多少？

A10. 8051 堆栈与 MOV SP

```
; 初始：SP=07H，A=3CH，08H=00H，30H=55H
MOV SP,#2FH
PUSH ACC
PUSH 30H
POP 31H
POP 32H
```

要求：写出最终 SP、30H、31H、32H；指出 PUSH/POP 的入栈出栈顺序。

B 组：Cortex-M3/M4 MOV / MOVW / MOVT / LDR 伪指令

B1. Cortex-M：MOV 是寄存器传送，不是取内存

```
; 初始：R0=0x20000000，内存[0x20000000]=0x12345678，R1=0
MOV R1,R0
; 与 LDR R2,[R0] 对比
```

要求：写出 R1 的值；若执行 LDR R2,[R0]，R2 是什么？

B2. Cortex-M：MOVS 对标志位的影响

```
; 初始：APSR 的 Z=0，N=0
MOVS R0,#0
MOVS R1,#0x80
```

要求：写出两条指令后 R0、R1，并说明 Z/N 如何变化。

B3. Cortex-M：大常数不能想当然 MOV

```
MOV R0,#0x12345678
```

要求：说明这条在很多汇编器/编码场景下为什么可能不成立；写出两种可靠装载方式。

B4. Cortex-M：MOVW/MOVT 顺序

```
MOVT R4,#0x1234
MOVW R4,#0x5678
```

要求：写出最终 R4，并说明如果要得到 0x12345678 应如何改。

B5. Cortex-M：MOVW 清高 16 位，MOVT 保低 16 位

```
; 初始：R5=0xAAAA5555
MOVT R5,#0x1234
MOVW R5,#0x00EF
```

要求：写出每步后的 R5。

B6. Cortex-M：LDR =const 是伪指令

```
LDR R0,=0x000000FF
LDR R1,=0x12345678
```

要求：说明汇编器可能分别生成什么形式；为什么不能把它等同于普通 LDR [Rn]。

B7. Cortex-M：内存映射寄存器初始化模板

```
; 目标：把 0x00000005 写入地址 0x40021018
LDR  R0,=0x40021018
MOVS R1,#5
STR  R1,[R0]
```

要求：说明三条指令分别完成什么；最终哪个地址被写入什么值？

B8. Cortex-M：小端序和 STR 结果

```
; 初始：R0=0x20000000
MOVW R1,#0x5678
MOVT R1,#0x1234
STR  R1,[R0]
```

要求：写出 R1 以及地址 0x20000000 到 0x20000003 的字节内容。

C 组：混合纠错与接口综合

C1. 混合纠错：把非法 8051 代码改成可汇编

```
; 目标：把 R3 的值复制到 R2，并把内部 RAM[40H] 复制到内部 RAM[41H]
MOV R2,R3
MOV @R0,@R1
```

要求：给出一版可汇编且语义清楚的 8051 改写；可使用 A 作为中转。

C2. 混合纠错：把 ARM 思维误用到 8051

```
; 目标：把外部 XDATA[1234H] 读到 A，再保存到内部 RAM 30H
MOV R0,#1234H
MOV A,[R0]
MOV 30H,A
```

要求：指出错误，并写出 8051 正确写法。

C3. 接口类小综合：8051 外设端口映射

```
; 外设数据端口映射到 XDATA 9000H，控制端口映射到 XDATA 9001H
; 要求：先写控制字 80H，再写数据 3CH
```

要求：写出一段 8051 汇编，要求使用 DPTR 和 MOVX。

C4. Cortex-M 小综合：拆装常数与写寄存器

```
; 目标：把 0xA5A55A5A 写入内存映射寄存器 0x40010000
```

要求：写出不用 literal pool 的 Cortex-M3/M4 汇编片段。

C5. 高频问答：判断 MOV 是否改变判断条件

```
; Cortex-M
CMP  R0,#0
MOV  R1,R2
BEQ  label

; 8051
CLR  C
```

```
MOV  A,#0FFH
JC   somewhere
```

要求：分别说明跳转条件是否会被中间的 MOV 改变。

C6. 综合跟踪：8051 银行、间接寻址、直接地址混合

```
; 初始：PSW=08H, 08H=20H, 20H=11H, 21H=22H, 30H=33H
MOV A,@R0
MOV 21H,A
MOV 08H,#30H
MOV A,@R0
MOV 22H,A
```

要求：写出 A、20H、21H、22H、08H 的最终值。

四、答案与详解

解析重点不是只给结果，而是训练“先判空间、再判数据流、最后判副作用”的顺序。考试遇到变形题，可以直接套这个顺序。

A1. 解析

A1 最终 A=34H，30H=56H，31H=78H，32H=34H。

关键是 MOV A,31H 在 31H 被改成 78H 之前已经把旧值 34H 送入 A；最后 MOV 32H,A 保存的是 34H。

A2. 解析

PSW=08H 表示 RS1:RS0=01，当前使用寄存器银行 1，所以 R0 对应物理 08H，R1 对应物理 09H。

执行 MOV R0,#30H 后 08H=30H；MOV R1,#31H 后 09H=31H。随后 MOV 00H,#55H 只改物理 00H，不改当前 R0。最后 MOV 08H,#66H 改了当前 R0 的物理单元，所以当前 R0=66H，R1=31H；物理 00H=55H，08H=66H。

A3. 解析

最终 A=56H，40H=56H，41H=12H。

@R0 表示把 R0 的内容 40H 当作内部 RAM 地址。第一次 MOV A,@R0 读 40H 的 12H；MOV 41H,A 保存 12H；MOV @R0,#56H 把 56H 写入 40H；最后 MOV A,40H 读回 56H。

A4. 解析

合法：② MOV A,R3，④ MOV 35H,@R1，⑤ MOV DPTR,#1234H，⑦ MOV C,20H，⑧ MOV 20H,C。非法：①、③、⑥。

① 可改为 MOV A,R3 / MOV R2,A，或用直接地址方式转存。③ 没有 @Ri 到 @Ri 的单条 MOV，可用 A 中转。⑥ DPTR 只能直接装 16 位立即数，不能 MOV DPTR,A；若要由两个字节组成，应分别处理 DPH/DPL 或用合适中转。

A5. 解析

最终 20H=22H，21H=22H，22H=22H。

汇编写法目的在前：MOV 20H,21H 是把 21H 的内容送到 20H。容易错在 8051 的 direct,direct 机器码中源/目的操作数字节顺序与书写顺序不直观。

A6. 解析

最终 C=1，位 20H=1，位 21H=1。

MOV C,20H 使 C=1；MOV 21H,C 使 21H=1；再 MOV C,21H 仍为 1；MOV 20H,C 保持 20H=1。这里是位地址传送，不是字节地址 20H 的整体复制。

A7. 解析

最终 DPTR=2000H，A=55H。

MOVC A,@A+DPTR 使用代码存储器地址 DPTR+A，也就是 2000H+02H=2002H，表项是 55H。MOVC 读的是 code space，不是内部 RAM 或外部 XDATA。

A8. 解析

最终 DPTR=8000H，A=5AH，内部 RAM 30H=5AH。

MOVX 访问外部数据存储器 XDATA；MOV 30H,A 是内部 RAM 直接地址写入。MOV、MOVC、MOVX 的地址空间是这类题的核心。

A9. 解析

P1=0F0H；随后 MOV A,90H 按直接地址读 90H，A=0F0H；30H=0F0H。

考试中若题干把 90H 明确说明为 P1 SFR，就按 SFR 直接地址处理。不要把 direct 90H 和 @R0 间接到 90H 混为一谈。

A10. 解析

MOV SP,#2FH 后 SP=2FH。PUSH ACC：SP 先加到 30H，再把 A=3CH 写入 30H。PUSH 30H：SP 到 31H，把 30H 的内容 3CH 再写入 31H。

POP 31H 先从 31H 读出 3CH 写到 31H，然后 SP 变 30H；POP 32H 再从 30H 读出 3CH 写到 32H，SP 变 2FH。最终 SP=2FH，30H=3CH，31H=3CH，32H=3CH。

B1. 解析

MOV R1,R0 后 R1=0x20000000，它只是复制寄存器值。

若执行 LDR R2,[R0]，才会把 R0 当地址去访问内存，所以 R2=0x12345678。方括号是 Cortex-M 访存题的第一分界线。

B2. 解析

MOVS R0,#0 后 R0=0，Z=1，N=0。

MOVS R1,#0x80 后 R1=0x00000080，Z=0，N=0，因为 32 位结果最高位 bit31 仍为 0。若装入的是 0x80000000 这类值，N 才会为 1。

B3. 解析

R0 是 32 位并不意味着一条 MOV 能编码任意 32 位立即数。Thumb-2 对 MOV 立即数有编码限制；很多大常数需要其他方式。

可靠写法一：MOVW R0,#0x5678 / MOVT R0,#0x1234。可靠写法二：LDR R0,=0x12345678，由汇编器选择 literal pool 或可编码指令。

B4. 解析

最终 R4=0x00005678。

因为 MOVT 先把高 16 位设为 0x1234，但随后 MOVW R4,#0x5678 会写低 16 位并清高 16 位。要得到 0x12345678，应写 MOVW R4,#0x5678，再 MOVT R4,#0x1234。

B5. 解析

初始 R5=0xAAAA5555。MOVT R5,#0x1234 后 R5=0x12345555，因为 MOVT 改高 16 位、保留低 16 位。

MOVW R5,#0x00EF 后 R5=0x000000EF，因为 MOVW 写低 16 位并清零高 16 位。

B6. 解析

LDR R0,=0x000000FF 可能被汇编器优化成 MOV/MOVS 一类立即数装载；LDR R1,=0x12345678 可能生成 MOVW/MOVT，也可能从 literal pool 取常数。

它是伪指令，语义是把常数装入寄存器；普通 LDR Rn,[Rm] 的语义是从 Rm 指向的内存地址取数据。

B7. 解析

LDR R0,=0x40021018 装入外设寄存器地址；MOVS R1,#5 准备要写的值；STR R1,[R0] 把 0x00000005 写到地址 0x40021018。

MOV 没有直接访问外设，真正写外设的是 STR。

B8. 解析

MOVW/MOVT 后 R1=0x12345678。

Cortex-M 通常按小端序存储字，所以 0x20000000=78H，0x20000001=56H，0x20000002=34H，0x20000003=12H。

C1. 解析

复制 R3 到 R2：MOV A,R3 / MOV R2,A。

若 R0、R1 分别保存源和目的内部 RAM 地址，则 @R1,@R0 不能直接写。可写：MOV A,@R0 / MOV @R1,A。若题目明确要 RAM[40H] 到 RAM[41H]，可写 MOV 41H,40H。

C2. 解析

错误点：8051 的 R0 是 8 位，不能 MOV R0,#1234H；8051 也没有 MOV A,[R0] 这种 ARM 风格方括号寻址；外部 XDATA 应使用 MOVX。

正确写法：MOV DPTR,#1234H / MOVX A,@DPTR / MOV 30H,A。

C3. 解析

可写：MOV DPTR,#9001H / MOV A,#80H / MOVX @DPTR,A / MOV DPTR,#9000H / MOV A,#3CH / MOVX @DPTR,A。

控制字和数据都在外部数据空间映射地址上，所以写出动作必须是 MOVX @DPTR,A。

C4. 解析

不用 literal pool 可写：MOVW R0,#0x0000 / MOVT R0,#0x4001 / MOVW R1,#0x5A5A / MOVT R1,#0xA5A5 / STR R1,[R0]。

R0 装地址 0x40010000，R1 装数据 0xA5A55A5A，最后 STR 访问内存映射寄存器。

C5. 解析

Cortex-M 片段中 CMP 设置条件标志；普通 MOV R1,R2 不带 S，不改变 Z，所以 BEQ 是否跳转由 CMP R0,#0 的结果决定。

8051 片段中 CLR C 后 C=0；MOV A,#0FFH 不改变 C，所以 JC 不跳转。若是 MOV C,bit，则会覆盖 C，需要另算。

C6. 解析

PSW=08H 使用银行 1，因此当前 R0 是物理 08H。初始 08H=20H，所以第一次 MOV A,@R0 读内部 RAM[20H]=11H，A=11H；MOV 21H,A 后 21H=11H。

MOV 08H,#30H 改的是当前 R0 的物理位置，所以 R0 变成 30H。第二次 MOV A,@R0 读内部 RAM[30H]=33H，A=33H；MOV 22H,A 后 22H=33H。最终 A=33H，20H=11H，21H=11H，22H=33H，08H=30H。

五、临考深挖：常见变形题怎么拆

看到寄存器名先问银行

8051 的 R0-R7 会随 PSW.RS1:RS0 切换；直接写 00H-1FH 是物理地址，不随银行名字自动变化。

看到 80H-FFH 先问寻址方式

直接地址可能是 SFR；间接地址 @R0/@R1 则走内部 RAM 间接空间。题目把 90H 写成 P1 时，通常考 direct 到 SFR。

看到大立即数先问编码

Cortex-M 的寄存器宽度和 MOV 立即数可编码范围是两件事。大常数优先考虑 MOVW/MOVT 或 LDR =const。

看到端口初始化先分三步

装地址、装数据、真正写出。8051 写外部映射端口通常 MOVX；Cortex-M 写内存映射寄存器通常 STR。

看到条件跳转先回溯最近一次改标志的指令

普通 MOV 多数情况下不改条件标志；MOVS、CMP、TST、加减类才常常决定后续分支。8051 中 MOV A,#data 不会把 C 变成 1。

看到查表先确认地址空间

8051 的 MOVC 取 code space，MOVX 取 external data space，MOV 取内部数据/SFR。助记符差一个字母，答案空间完全不同。

六、最后十分钟自测清单

- 8051：能否一眼判断 MOV A,@R0、MOV A,40H、MOVX A,@DPTR、MOVC A,@A+DPTR 的地址空间？
- 8051：能否解释为什么 MOV R2,R3 不作为标准单条指令使用？
- 8051：能否在 PSW=08H/10H/18H 时写出 R0 的物理地址？
- Cortex-M：能否说明 MOV R1,R0 与 LDR R1,[R0] 的差别？
- Cortex-M：能否不用 LDR =const 写出装入 0x12345678 的代码？
- 接口题：能否把“地址、数据、写出指令”三件事分开写？